Old Recommender Systems vs. TIGER

A full, example-by-example walkthrough — written from our own Q&A, no ML background assumed

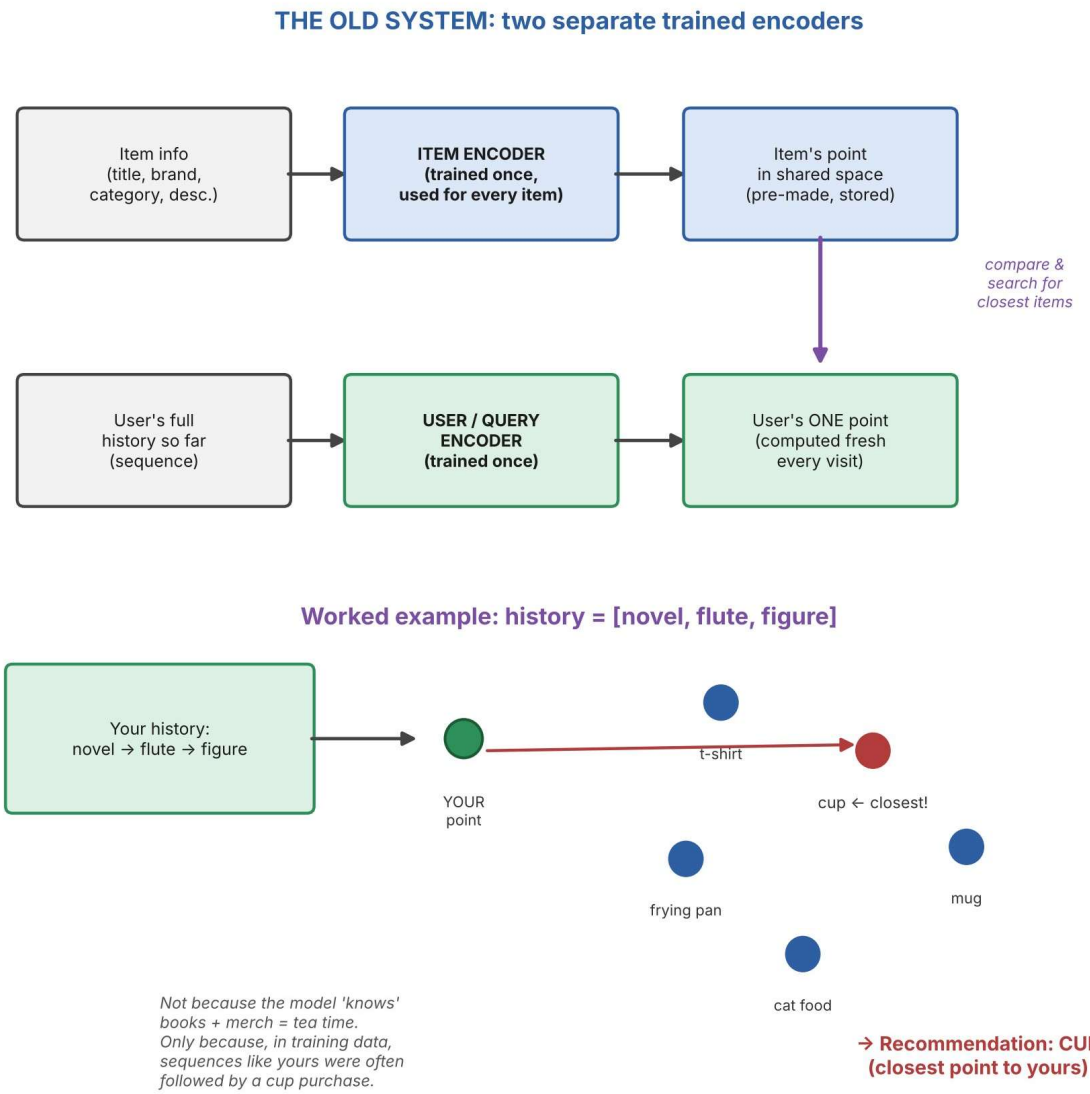
In one sentence: the old way turns everything into points in space and SEARCHES for the nearest one; TIGER turns everything into meaningful ID codes and WRITES the answer directly, like autocomplete.

PART 1 — The old system (search-based)

1.1 The big picture

Every item in the catalog (and every user) gets turned into a **point** in a big invisible space — think of GPS coordinates, except instead of 2 numbers (latitude, longitude) each point has hundreds of numbers. A point made this way is called an **embedding**. Two separate trained neural networks do the turning-into-points job:

- the **item encoder** — placed once per catalog item, in advance, and stored on a kind of shelf
- the **user/query encoder** — recomputes the user's single point fresh, every time, from their current history



The two encoders, and a full worked example: history = [novel, flute, figure] → closest catalog point = cup.

1.2 Walking through our example

Say your basket history is **[novel, flute, figure]**. The user encoder turns that whole sequence into ONE point. The system then looks at the pre-made shelf of item points — t-shirt, cup, frying pan, mug, cat food — and measures the distance from your point to each of them. Whichever catalog point sits physically closest wins. In our example that's the **cup**.

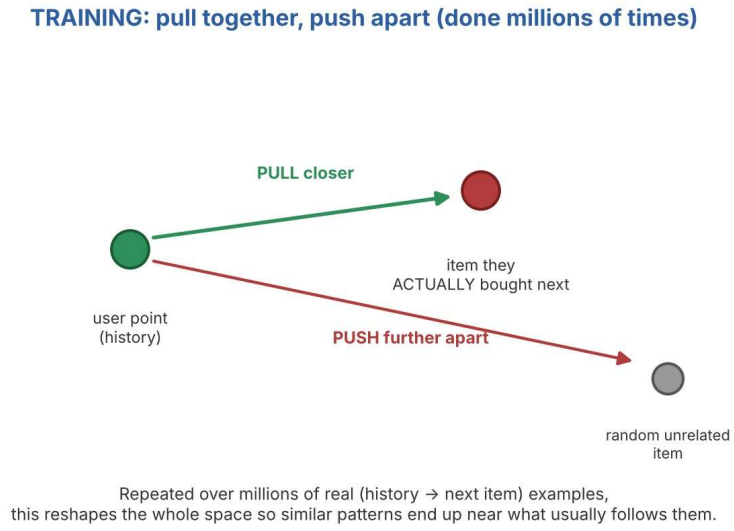
Important: the model doesn't “understand” that books + a flute + a figure suggest someone who likes cosy tea-time vibes. It only knows that, across millions of other real shoppers in the training data, sequences that looked like yours were very often followed by a cup purchase. It is pattern-matching on history, not reasoning about taste.

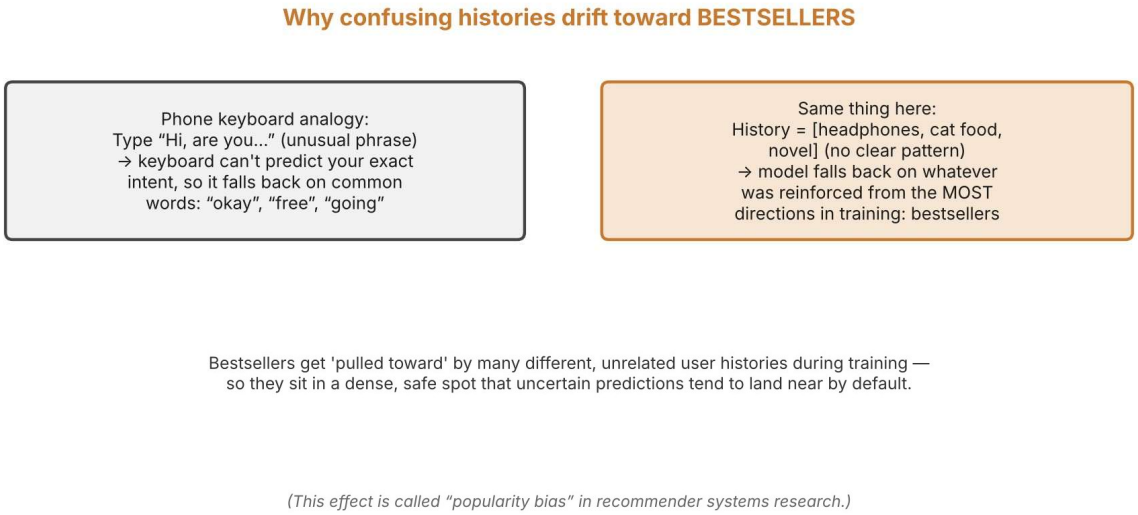
1.3 How the two encoders actually get trained

Before any of this works, both encoders have to be trained on millions of real (history → what the user bought next) examples. Training works like a tug-of-war:

- **Pull together:** the user's point and the item they ACTUALLY bought next are dragged closer to each other in the space
- **Push apart:** the user's point and some random, unrelated item are pushed further away

Repeated over and over, across huge amounts of real shopping data, this slowly reshapes the whole space so that “what usually comes next” ends up physically close together — without anyone hand-labelling what the categories mean.





Top: the pull/push training mechanic. Bottom: why a confusing, scattered history tends to drift toward bestsellers — same idea as a keyboard falling back on common words when it can't predict your exact sentence.

1.4 The catch with this approach

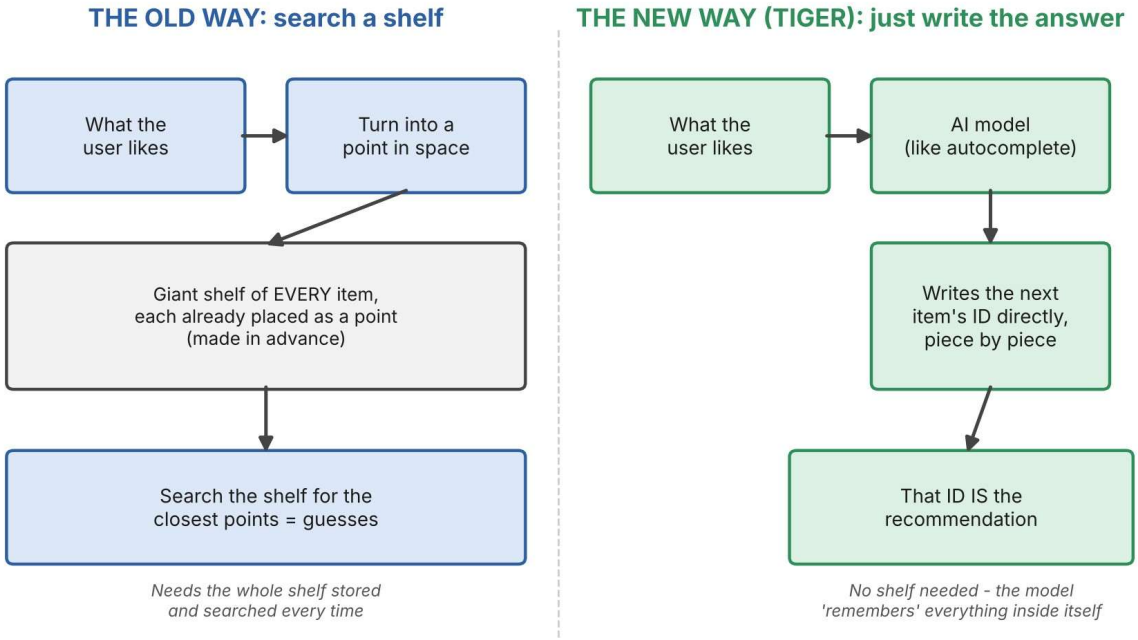
This system only works as well as its shelf of pre-made item points. Two consequences follow directly, and TIGER was built specifically to fix both of them (see Part 3):

- A brand-new item with zero sales has no reliable point yet — the “cold start” problem
- An uncertain / scattered user history tends to collapse toward safe, generic bestsellers, because theysit in a dense, frequently-reinforced part of the space

PART 2 — TIGER: generate, don't search

2.1 The core shift

TIGER (Transformer Index for GEnerative Recommenders) throws out the shelf entirely. Instead of placing points and searching for the nearest one, it trains a Transformer (the same family of model behind modern text autocomplete) to **directly write out** the ID of the next item, piece by piece — the same way your phone keyboard writes out the next word it thinks you want.



Side-by-side recap: search-the-shelf (old) vs. write-the-answer (TIGER).

2.2 What is a Semantic ID?

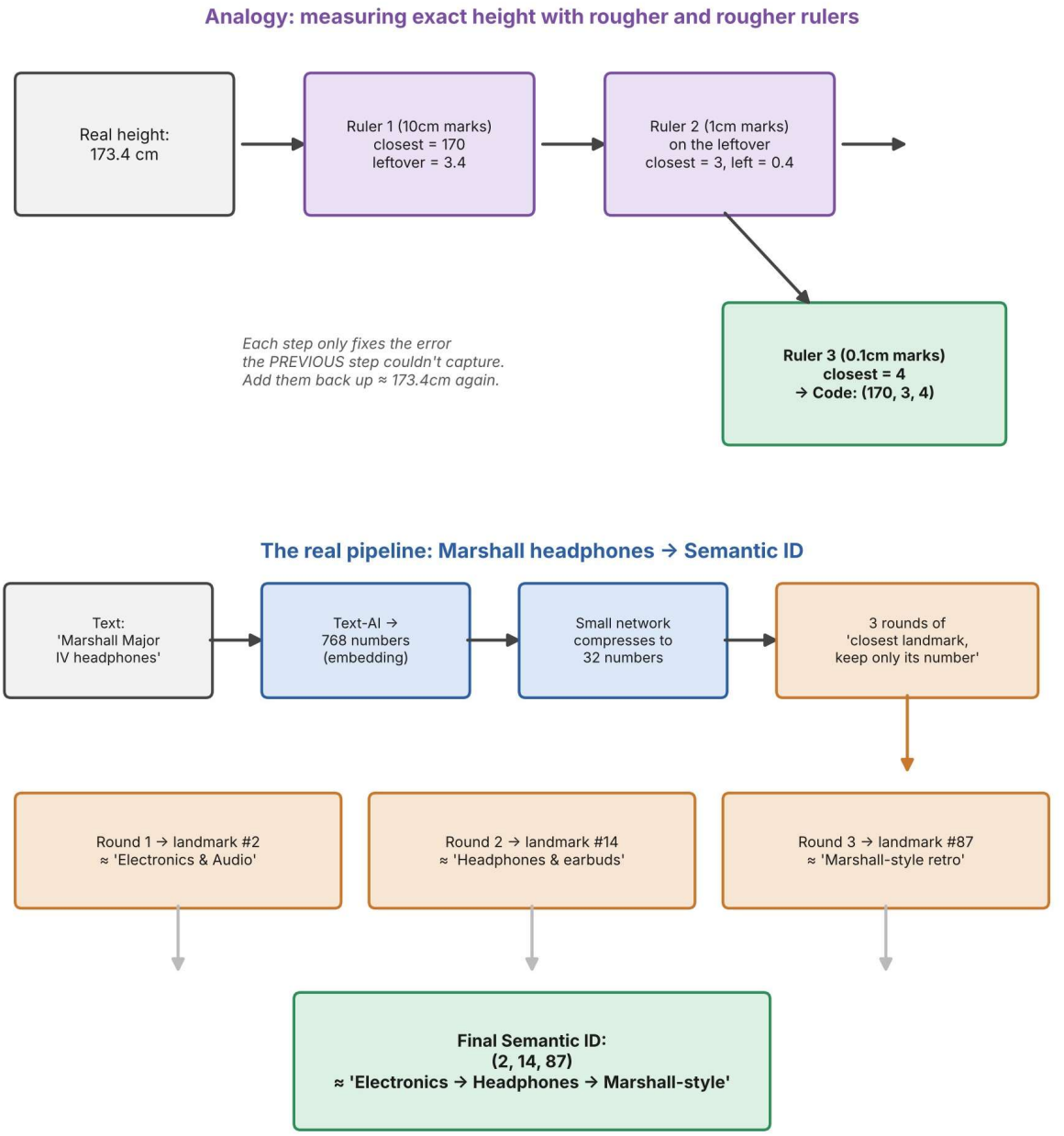
A Semantic ID is a short code (3 numbers, occasionally 4) that stands in for one catalog item — but unlike a random serial number, it's built directly from the item's real text (title, brand, category, description), so similar items naturally get similar codes. It also comes out **hierarchical**: the first number tends to capture the coarse category, and later numbers narrow it down to finer and finer detail. Nobody designs this hierarchy by hand — it just emerges from how the compression process works, and you can only see it afterwards by checking which real items end up sharing which numbers.

2.3 How a Semantic ID is actually built (RQ-VAE)

The building block is called **residual quantization**. It sounds complicated, but it's the same idea as measuring your exact height with rulers of decreasing coarseness:

> *Real height = 173.4cm. Ruler 1 (10cm marks) gets you to the closest mark: 170, leaving a 3.4cm leftover. Ruler 2 (1cm marks) measures THAT leftover: closest = 3, leaving 0.4 left. Ruler 3 (0.1cm marks) measures the final leftover: closest = 4. Final code: (170, 3, 4). Add them back up and you're back to ≈173.4cm. Each ruler only ever fixes the error the previous one couldn't capture — that's "residual" quantization.*

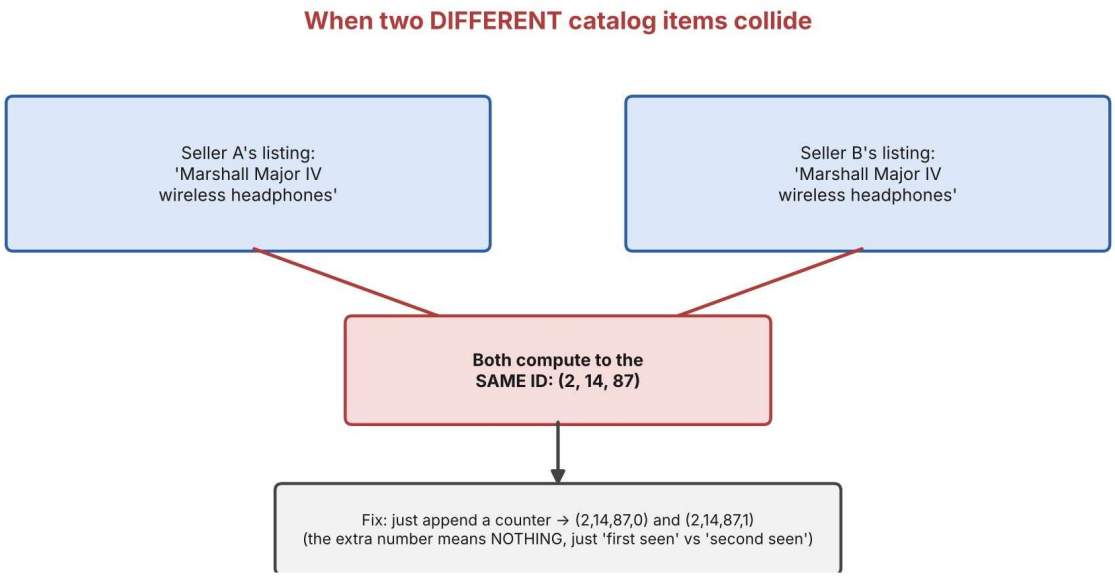
This is NOT ordinary division or a remainder calculation — it's always "find the closest match among a fixed set of landmark options," using multi-dimensional distance (the many-numbers version of Pythagoras' theorem, same idea as comparing GPS coordinates). And critically, at every round only the landmark's **index number** is kept — like only writing down "House #7 on Elm Street" instead of its full street address — which is exactly where the compression (768 numbers → 32 numbers → a 3-number code) comes from.



Top: the ruler/height analogy for residual quantization. Bottom: the real pipeline, worked through with Marshall headphones as the example item.

2.4 When two different items collide

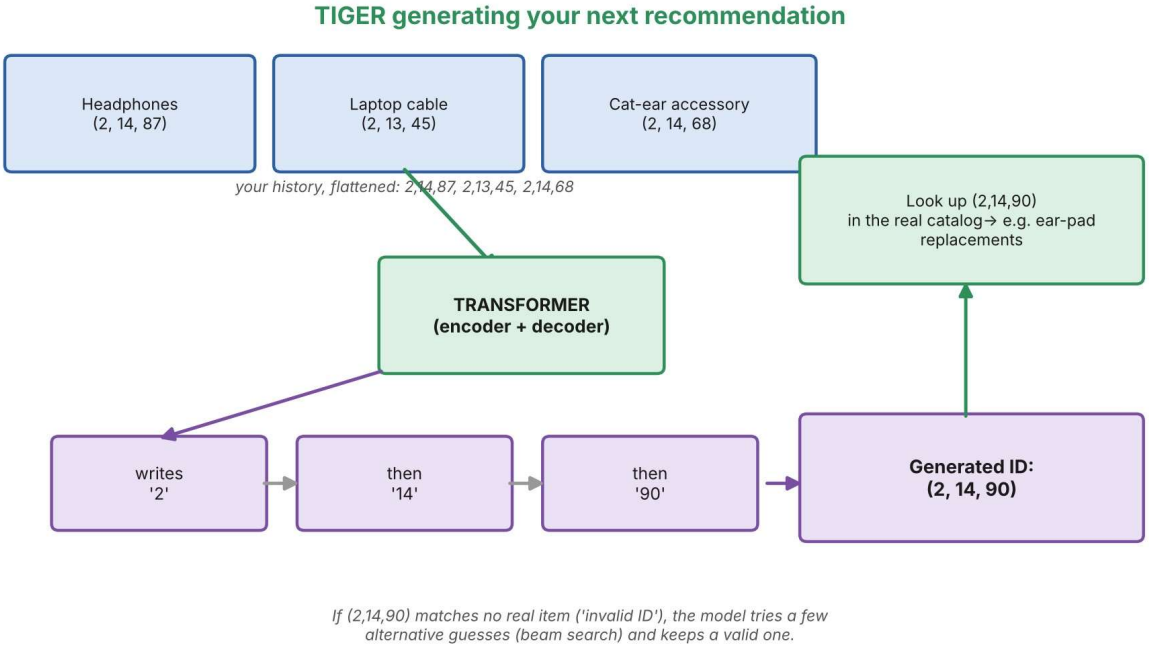
Sometimes two genuinely DIFFERENT catalog items (not two units of the same item — that's not a collision) end up computing to the exact same 3-number code, for example two different sellers both listing “Marshall Major IV wireless headphones.” When that happens, a 4th number is appended purely as a tie-breaker. It carries **no meaning at all** — it does not encode colour, battery life, or any real attribute. It only reflects which one the system happened to process first.



Two different listings colliding on the same 3-number ID, resolved with a meaningless 4th tie-breaker number.

2.5 Generating the next recommendation

Once every item has a Semantic ID, a user's whole shopping history becomes one long sequence of numbers (their items' IDs, one after another). An encoder-decoder Transformer is trained to predict the next ID, **one number at a time**, exactly like predicting the next word in a sentence — each number it writes is conditioned on everything written before it. The finished generated ID is then looked up in a table that maps valid Semantic IDs back to real catalog items.



Worked example: history = headphones, laptop cable, cat-ear accessory → the model writes a new ID number-by-number → looked up as a real product.

This really is the autocomplete analogy in full: if you often type “hi, how are you,” your keyboard predicts that pattern confidently. If you start typing “hi, are you dumb,” it has no strong matching pattern, so it falls back on common, generic continuations like “are you okay” or “are you free.” A scattered, unclear shopping history produces the exact same effect — the model falls back on safe, frequently-seen choices (i.e. bestsellers), for the same underlying reason as in the old system (section 1.3/1.4). Occasionally the generated ID doesn't match any real item (an “invalid ID”) — in the paper this happens for roughly

0.1%–1.6% of top-10/20 predictions, and is handled by generating a few candidate IDs at once (“beam search”) and keeping a valid one.

PART 3 — Why this design is actually worth it

3.1 Fixing cold start

The old system can only place an item's point using patterns learned from real purchase history — so a brand-new item with zero sales has nowhere reliable to sit, and barely ever gets recommended. TIGER builds Semantic IDs from an item's TEXT alone (title, brand, description), which is available on day one. A new item lands near similar existing items automatically, with zero sales required.



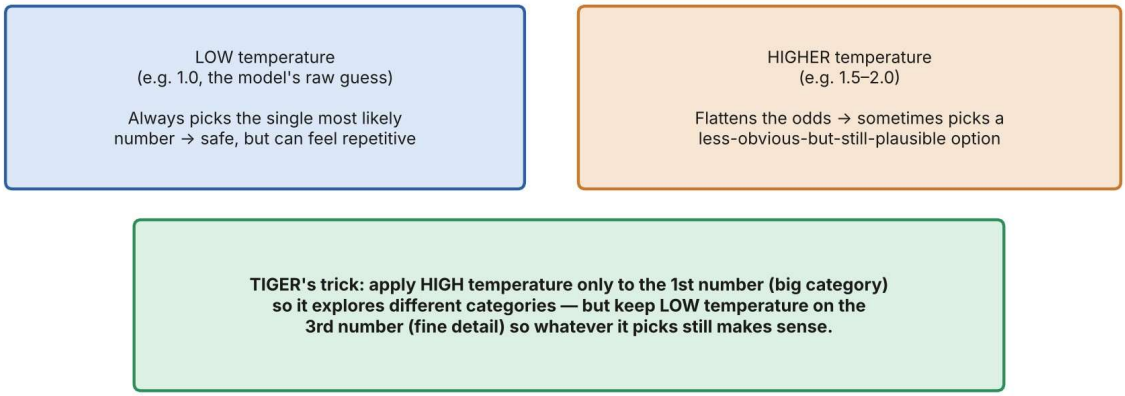
Worked example: a brand-new "Marshall Major V" launch gets an ID almost identical to the existing "Marshall Major IV" from its description alone, and can be recommended to fans immediately.

3.2 Controlling boring vs. interesting results (temperature)

When the model writes out an ID, it's really choosing from a probability distribution over possible next numbers. A setting called **temperature** reshapes that existing distribution — it does not invent new options out of nowhere. Low temperature ≈ always pick the single most likely number (safe, can feel repetitive). Higher temperature ≈ flatten the odds so less-dominant-but-still-plausible options get picked sometimes.

TIGER's actual trick is applying this **selectively per position**: higher temperature only on the 1st number (the coarse category) so it genuinely explores different categories sometimes, while keeping lower temperature on the later, fine-detail numbers so whatever category gets explored still makes sense once you're in it. It is one single global setting applied to the model's own learned odds — you can't hand-pick or inject a specific category yourself.

Controlling 'safe vs. interesting' recommendations



Paper's own measured results (more variety as temperature rises):

Temperature	Variety score (Entropy@10)
1.0	0.76
1.5	1.14
2.0	1.38

How temperature trades off safe vs. varied picks, applied unevenly across the 3 ID positions, with the paper's own measured entropy numbers.

3.3 The bottom line

Because items are represented by MEANING (built from real text) instead of a random serial number, the same underlying design solves two separate problems at once: brand-new items can be recommended immediately, and the system has a built-in, adjustable dial for how safe vs. adventurous its suggestions are — all without needing a giant pre-built shelf of every item in the catalog.

Quick glossary

Embedding	A point made of many numbers (not just one) representing an item or a user — like GPS coordinates with hundreds of values instead of 2.
Dual-encoder / nearest-neighbour search	The old system: two separately-trained encoders place everything as points, then search finds the closest ones.
Transformer	The type of neural network behind both modern text autocomplete and TIGER; predicts the next token conditioned on everything before it.
Semantic ID	A short, meaning-based code (usually 3-4 numbers) standing in for one catalog item, built from its real text content.
RQ-VAE / residual quantization	The ruler-by-ruler process that compresses a big embedding down into a short Semantic ID, one “closest landmark” round at a time.

Codebook / landmark	A fixed list of reference points at each round of quantization; only the index of the CLOSEST one is kept.
Collision	When two different items end up with the identical Semantic ID; fixed with a meaningless 4th tie-breaker number.
Cold start	The problem of recommending an item with little or no interaction history.
Temperature	A setting that reshapes (not invents) the model's existing probability distribution over what to generate next; higher = more variety.
Popularity bias	The tendency of both old and new systems to over-recommend bestsellers when a user's history doesn't clearly match a pattern.
Recall@K / NDCG@K	Standard scoring methods for whether the correct next item showed up in the model's top-K guesses, and how highly it was ranked.